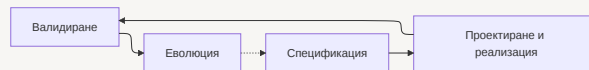


24. Модели и дейности на софтуерния процес. Гъвкави методи за разработване на софтуер. Качество на софтуера.

1. Софтуерен процес

Софтуерният процес (software process) е структуриран набор от дейности, необходими за разработването на софтуерна система. Различните организации използват различни процеси, но повечето включват четири фундаментални дейности.

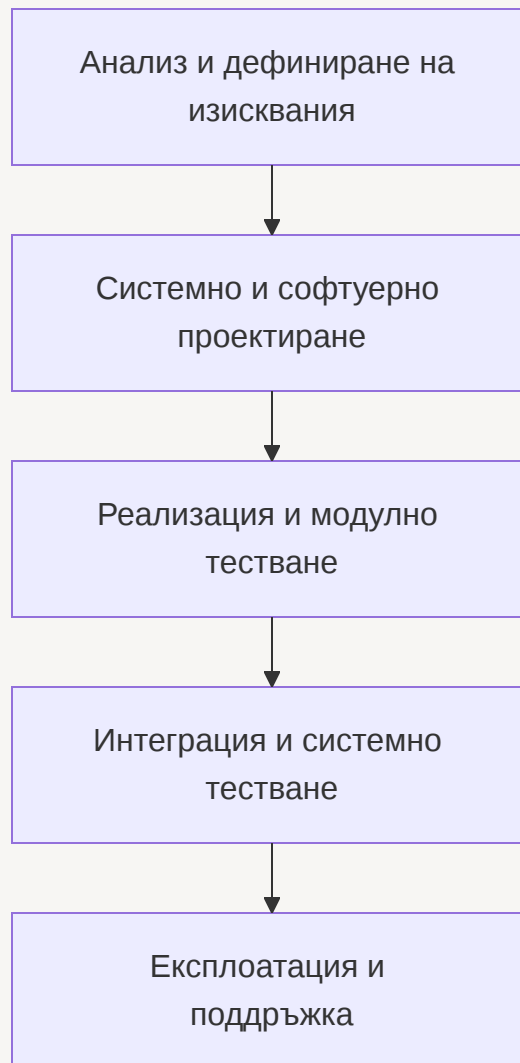
1. **Спецификация (Specification)** - дефиниране на функционалностите на софтуера и ограниченията върху неговата работа.
2. **Проектиране и реализация (Design and Implementation)** - създаване на софтуер, който отговаря на спецификацията.
3. **Проверка и валидиране (Validation)** - проверка дали софтуерът върши това, което клиентът желае.
 - a. **Верификация (Verification):** "Строим ли продукта правилно?" - проверява дали софтуерът съответства на специфицираните изисквания (прегледи на код, статичен анализ).
 - b. **Валидация (Validation):** "Строим ли правилния продукт?" - проверява дали софтуерът отговаря на реалните нужди на клиента (dynamic testing, acceptance testing).
4. **Еволюция / Поддръжка (Evolution)** - промяна на софтуера, за да отговаря на променящите се изисквания на клиента.



2. Модели на софтуерния процес

2.1 Водопаден модел (Waterfall)

Класически, планово-ориентиран модел - фазите се изпълняват последователно, а изходът от една фаза е вход за следващата.



Предимства: Ясна документация на всеки етап; лесен за планиране и контрол; подходящ при стабилни изисквания.

Недостатъци: Трудно връщане към предходна фаза; клиентът вижда работещ продукт едва накрая; неподходящ при неясни/динамични изисквания.

2.2 Инкрементална разработка (Incremental Development)

Спецификацията, разработката и валидирането се преплитат - системата се изгражда чрез последователни версии (инкременти), всяка от които добавя функционалност.

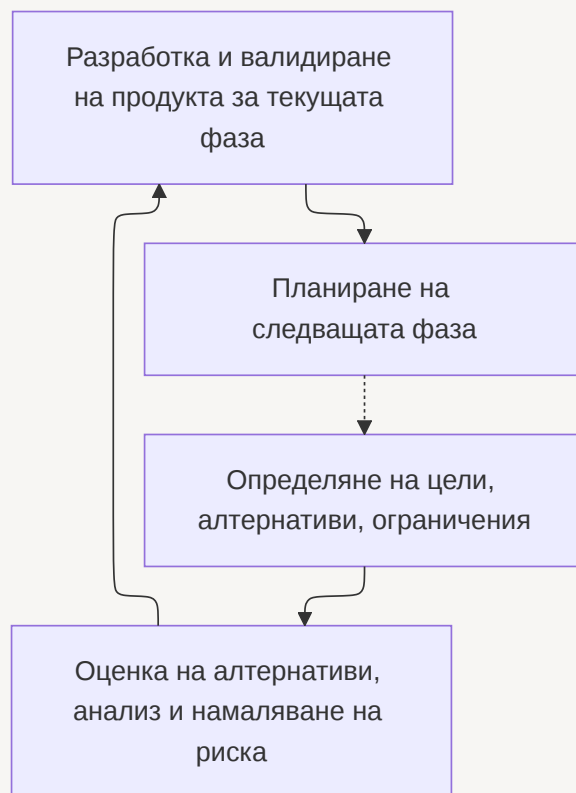


Предимства: По-бърза видима стойност за клиента; по-лесна адаптация към промени; понижен риск от цялостен провал.

Недостатъци: Процесът е по-труден за наблюдение; структурата на системата може да се влоши при чести промени.

2.3 Спирален модел (Spiral model)

Итеративен, рисково-ориентиран модел - процесът е спирала, всеки цикъл минава през четири сектора.



Всяко завъртане увеличава мащаба и зрелостта на продукта. Подходящ при висока несигурност и риск.

3. Дейности на софтуерния процес

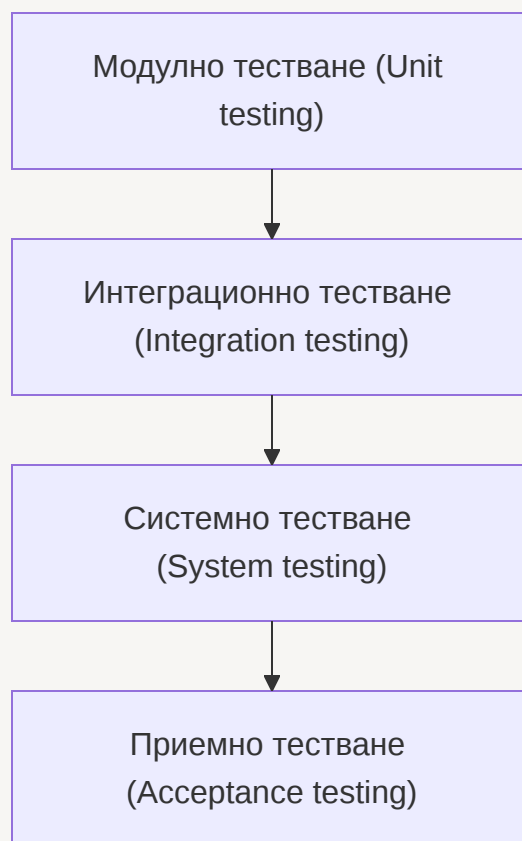
3.1 Инженеринг на изискванията

- **Проучване на осъществимостта (Feasibility study)** - техническа и финансова оправданост.
- **Извличане и анализ на изискванията** - комуникация с клиенти/потребители.
- **Спецификация на изискванията** - документ (SRS — Software Requirements Specification).
- **Валидиране на изискванията** - проверка за реалистичност, непротиворечивост, пълнота.

3.2 Проектиране и реализация

- **Архитектурно проектиране** - подсистеми и връзки между тях.
- **Проектиране на интерфейси** между компоненти.
- **Проектиране на компоненти и бази данни.**
- **Реализация (кодиране)** - превод на проекта в изпълним код.

3.3 Валидиране (Testing)



3.4 Еволюция (Software Evolution / Maintenance)

- **Коригираща (Corrective)** - отстраняване на дефекти.
- **Адаптивна (Adaptive)** - приспособяване към промени в средата.

- **Усъвършенстваща (Perfective)** - нова функционалност, по-добра производителност.
- **Превантивна (Preventive)** - подобряване на поддръжката преди да възникнат проблеми.

4. Гъвкави методи за разработване на софтуер (Agile Software Development)

4.1 Причини за възникване

Гъвкавите (Agile) методологии възникват като реакция срещу тромавите, силно документално-ориентирани класически процеси. Те се фокусират върху итеративната разработка, бързата доставка на работещ софтуер и адаптивността към промените.

През 2001 г. група разработчици формулира Agile Manifesto, поставящ приоритет на:

- хората и взаимодействието пред процеси и инструменти;
- работещия софтуер пред изчерпателна документация;
- сътрудничеството с клиента пред договорни преговори;
- реакцията на промяна пред следването на план.

4.2 Scrum

Scrum е най-популярния метод за управление на комплексни проекти.

Процесът е организиран в итерации с фиксирана продължителност (от 1 до 4 седмици), наречени **Спринтове (Sprints)**.

Ролите в Scrum:

- **Product Owner** - Представява интересите на бизнеса и клиента; управлява и приоритизира продуктовия backlog.
- **Scrum Master** - Фасилитатор, който премахва пречките пред екипа и осигурява спазването на Scrum процесите.

- **Developers** - Самоорганизиращ се, кросфункционален екип.

Артефакти:

- **Product Backlog** - Приоритизиран списък с всички изисквания, потребителски истории (User Stories) и задачи.
- **Sprint Backlog** - Списък със задачи, избрани за изпълнение през текущия спринт.
- **Increment** - Сборът от всички завършени елементи от беклога в края на спринта, отговарящи на Дефиницията за завършено (Definition of Done - DoD).

Церемонии (Събития):

- **Sprint Planning (Планиране на спринта)** - Определя се целта на спринта и се разписва Sprint Backlog.
- **Daily Scrum (Ежедневен Scrum)** - 15-минутна среща за синхронизация на екипа.
- **Sprint Review (Преглед на спринта)** - Демонстрация на готовия инкремент пред заинтересованите страни.
- **Sprint Retrospective (Ретроспектива)** - Анализ на изминалия спринт с цел подобряване на процесите и екипната работа.



4.3 Екстремно програмиране (Extreme Programming - XP)

XP е софтуерна методология, фокусирана върху техническото съвършенство и инженерните практики. Основни практики включват:

- **Програмиране по двойки (Pair Programming)** - Двама инженери работят на едно работно място (един пише код, друг го преглежда в реално време)

- **Разработка, водена от тестове (Test-Driven Development - TDD)** - Първо се пишат автоматизираните тестове, а след това кодът, който да ги покрие.
- **Непрекъснатата интеграция (Continuous Integration - CI)** - Кодът се интегрира в общото хранилище многократно на ден, придружен от автоматични тестове.
- **Рефакторинг (Refactoring)** - Постоянно подобряване на структурата на кода без промяна на външното му поведение.

4.4 Kanban

Визуализация на работния поток чрез табло с колони (To Do / In Progress / Done), ограничение на работата в прогрес (WIP limit), без фиксирани по продължителност итерации - за разлика от Scrum.

4.5 Гъвкави срещу планово-ориентирани методи

Критерий	Планово-ориентирани	Гъвкави
Изисквания	Фиксирани в началото	Променят се непрекъснато
Документация	Изчерпателна	Минимална
Клиент	Участва в началото/края	Постоянно въввлечен
Доставка	Еднократна, в края	Чести инкременти
Подходящ за	Големи, критични системи	Малки/средни екипи, динамични изисквания

5. Качество на софтуера

5.1 Определение

Качеството на софтуера е степента, в която системата отговаря на специфицираните изисквания и на имплицитните очаквания на потребителя.

5.2 Характеристики на качеството

- **Функционалност** - покрива ли необходимите функции.
- **Надеждност** - работа без откази.
- **Използваемост** - лекота на употреба и обучение.
- **Ефективност** - оптимално използване на ресурси.
- **Поддържаемост** - лекота на модификация.
- **Преносимост** - лекота на пренасяне в друга среда.

5.3 Осигуряване и контрол на качеството

- **Осигуряване на качеството (Quality Assurance, QA)**
 - **Процесно-ориентирано.** Фокусира се върху дефинирането и подобряването на процесите на разработка, методологиите и стандартите, за да се предотвратят дефекти още в ранен етап (превантивна дейност).
- **Контрол на качеството (Quality Control, QC)**
 - **Продуктово-ориентирано.** Фокусира се върху идентифицирането на дефекти в реално създадения софтуерен продукт (детективна дейност). Включва дейности като софтуерно тестване (компонентно, интеграционно, системно).

5.4 Дейности по осигуряване на качеството

- **Технически прегледи (Reviews)** - walkthrough, инспекции (напр) - систематична проверка на документи/код от екип, без изпълнение на програмата.
- **Стандарти** - вътрешнофирмени, национални, международни (ISO).
- **План за качество (Quality plan)** - цели и практики за качество на конкретен проект.